

Manuale Tecnico

TheKnife - Laboratorio Interdisciplinare B

Autori: Hamdar Youssef (753832, Como) - Dellatorre Federico (755856, Como)

Versione: 1.0 - Laboratorio Interdisciplinare B

Indice

1. Introduzione tecnica
2. Architettura client/server
3. Struttura package e classi
4. Database PostgreSQL
5. Schema ER e relazionale
6. Diagramma UML
7. Comunicazione e concorrenza
8. Accesso ai dati con DAO e JDBC
9. Sicurezza e validazione
10. Maven e generazione jar
11. Limiti e sviluppi futuri

1. Introduzione tecnica

Il sistema TheKnife Lab B è stato sviluppato in Java 21 e realizza una piattaforma client/server con interfaccia grafica Swing, persistenza su PostgreSQL e accesso ai dati tramite JDBC. Il sistema separa la GUI, la comunicazione client/server, la logica di gestione delle richieste e il livello DAO.

2. Architettura client/server

Il modulo clientTK contiene il client applicativo e la GUI. Il modulo serverTK contiene il server, che accetta connessioni TCP sulla porta configurata e crea un ClientHandler per gestire ciascuna richiesta. Il server comunica con PostgreSQL attraverso classi DAO.



Figura 1 - Architettura generale client/server.

3. Struttura package e classi

Package	Classe	Responsabilita
theknife.gui	TheKnifeGUI	Interfaccia grafica Swing e gestione menu per ruolo
theknife.clientTK	ClientTK	Invio richieste al server tramite socket
theknife.serverTK	ServerTK	Avvio server e ascolto connessioni
theknife.serverTK	ClientHandler	Parsing richieste e chiamata servizi/DAO
theknife	Utente, Ristorante, Recensione	Classi modello
theknife	UtenteDAO, RistoranteDAO, RecensioneDAO, PreferitoDAO	Accesso a PostgreSQL con JDBC

4. Database PostgreSQL

Il database dbTK contiene le informazioni persistenti dell'applicazione. Le tabelle principali sono utenti, ristoranti, recensioni e preferiti. Le password sono salvate in forma cifrata; le recensioni collegano autore e ristorante; i preferiti collegano cliente e ristorante.

5. Schema ER e relazionale

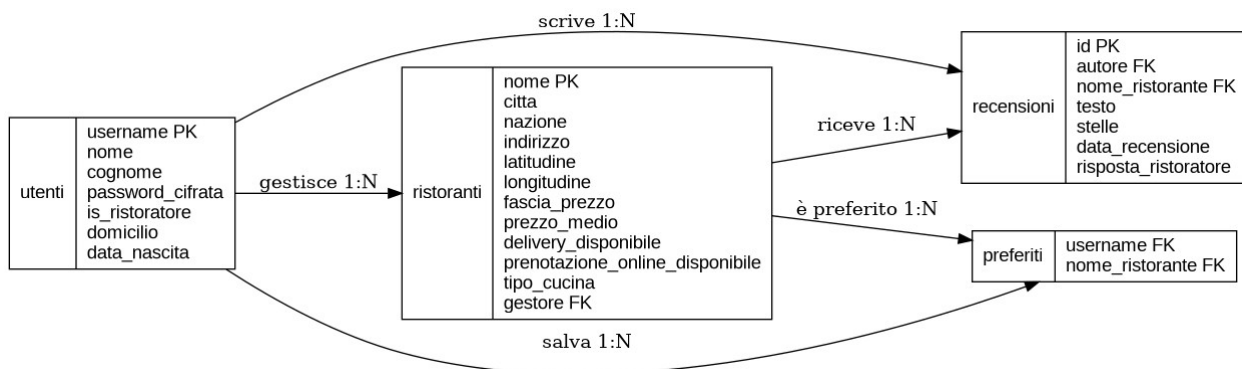


Figura 2 - Schema ER/logico semplificato del database.

Schema relazionale semplificato:

- utenti(username, nome, cognome, password_cifrata, is_ristoratore, domicilio, data_nascita)
- ristoranti(nome, citta, nazione, indirizzo, latitudine, longitudine, fascia_prezzo, prezzo_medio, delivery_disponibile, prenotazione_online_disponibile, tipo_cucina, gestore)
- recensioni(id, autore, nome_ristorante, testo, stelle, data_recensione, risposta_ristoratore)
- preferiti(username, nome_ristorante)

6. Diagramma UML

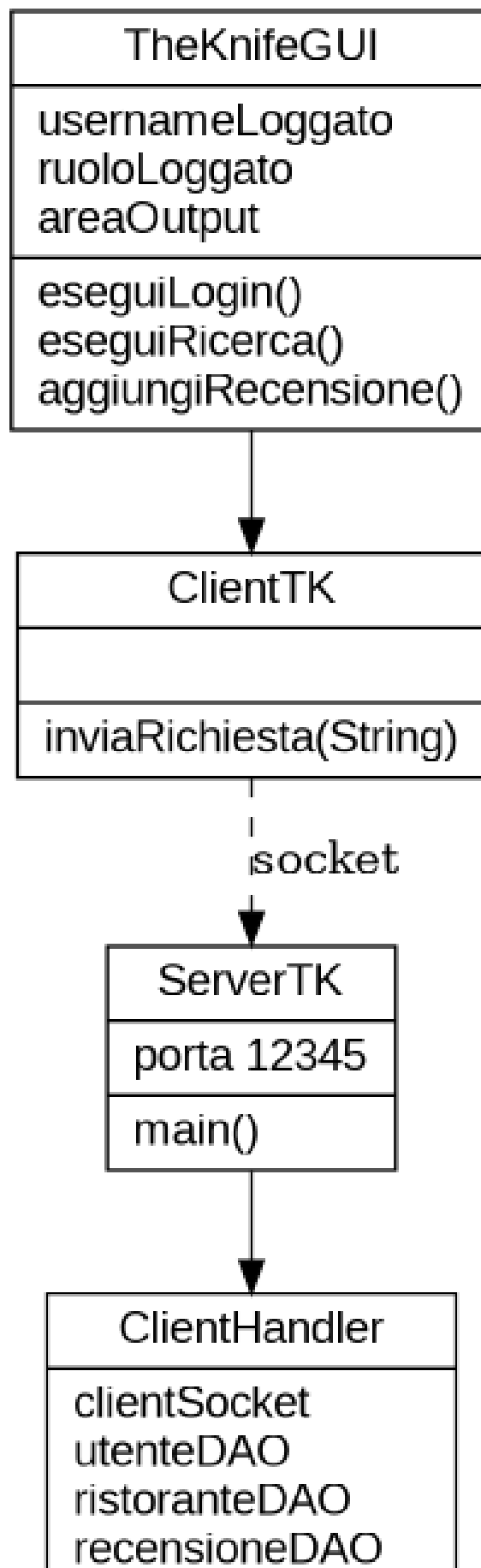


Figura 3 - Diagramma UML semplificato delle classi principali.

7. Comunicazione e concorrenza

La comunicazione tra client e server avviene tramite socket TCP. Il client invia richieste testuali con campi separati dal carattere |. Il server interpreta il comando e restituisce una risposta testuale. Per supportare più client, il server crea un handler separato per ogni connessione; in questo modo richieste di utenti diversi possono essere gestite in parallelo.

8. Accesso ai dati con DAO e JDBC

Le classi DAO incapsulano le query SQL e separano la logica applicativa dall'accesso al database. Ogni metodo apre una connessione tramite DatabaseConnection, prepara statement parametrizzati e restituisce oggetti modello o valori di esito.

- UtenteDAO: registrazione, ricerca utente e lettura utenti
- RistoranteDAO: inserimento e ricerca ristoranti, riepilogo recensioni
- RecensioneDAO: inserimento, modifica, eliminazione e risposta recensioni
- PreferitoDAO: aggiunta, rimozione e visualizzazione preferiti

9. Sicurezza e validazione

- Le password vengono cifrate tramite funzione di hash prima del salvataggio
- Le query usano PreparedStatement per ridurre il rischio di SQL injection
- Il server valida formato delle richieste, valori numerici, stelle tra 1 e 5, booleani true/false e data di nascita
- La GUI mostra pulsanti diversi a seconda del ruolo: cliente o ristoratore

10. Maven e generazione jar

Il file pom.xml definisce dipendenze e plugin Maven. Il comando `mvn clean package` genera gli eseguibili in target. Per la consegna sono stati prodotti due jar separati: `clientTK.jar` e `serverTK.jar`, copiati nella directory `bin/`.

11. Limiti e sviluppi futuri

- Miglioramento della grafica con JavaFX o framework dedicati
- Ricerca avanzata con combinazione di filtri più completa
- Gestione utenti e sessioni più avanzata
- Deployment server remoto invece di solo ambiente locale
- Gestione prenotazioni e notifiche